Explicación del diagrama de clases

Proyecto UrbanRoute (Red de Metro de Caracas) · Autor: Julio Reyes

Este documento explica cómo leer el diagrama de clases del proyecto y qué representa cada parte. El diagrama muestra las 11 clases del programa, sus atributos y métodos principales, y cómo se relacionan entre sí.

1. Cómo leer el diagrama

Las cajas (clases)

Cada caja es una clase y está dividida en tres partes:

1. **Nombre de la clase** (arriba). Si tiene algo entre «comillas angulares», es un estereotipo, por ejemplo `«extends JFrame»` indica que `InterfazUsuarioUI` hereda de la ventana de Swing.
2. **Atributos** (en el medio), con el formato `nombre: tipo`.
3. **Métodos** (abajo), con el formato `nombre(tipos de los parámetros): tipo que devuelve`.

El símbolo del inicio indica la visibilidad: `+` es público (otras clases lo pueden usar) y `-` es privado (solo se usa dentro de la clase). En las clases grandes solo se muestran los métodos más importantes; por ejemplo, en `Estacion` los getters y setters se resumen en una sola línea.

Los recuadros punteados (paquetes)

Las clases están agrupadas en paquetes, igual que en las carpetas del proyecto en NetBeans. Cada recuadro gris punteado es un paquete y su nombre aparece en la esquina superior izquierda.

Las flechas (relaciones)

Flecha	Nombre	Qué significa en este proyecto
Línea continua con punta abierta	Asociación	La clase tiene un atributo del otro tipo. Ej.: `AnalizadorRedUrbana` tiene el atributo `grafo`.
Línea continua con rombo negro	Composición	La clase crea y contiene a la otra; si la clase se destruye, la parte también. Ej.: `GrafoTransporte` crea su propia `TablaHashEstaciones`.
Línea continua con rombo blanco	Agregación	La clase guarda objetos que existen por sí solos. Ej.: la tabla hash guarda estaciones, pero no las crea.
Línea punteada con punta abierta	Dependencia	La clase usa a la otra dentro de un método (como parámetro, variable local o valor de retorno), pero no la guarda como atributo.

El rombo siempre va del lado de la clase "dueña" y la punta de flecha del lado de la clase que se usa. Los números junto a las flechas son las **multiplicidades**: `1` = exactamente uno, `0..*` = cero o muchos, `0..1` = ninguno o uno, `16..*` = dieciséis o más.

2. Organización por paquetes (capas)

El programa está dividido en cuatro capas. Cada capa solo conoce a las de abajo, así cada clase tiene una sola responsabilidad:

Paquete	Clases	Responsabilidad
`urbanroute`	Main	Punto de inicio: abre la ventana.
`urbanroute.vista`	InterfazUsuarioUI	Todo lo que ve el usuario: botones, formularios, mensajes y el dibujo del grafo.
`urbanroute.control`	GestorArchivoCSV, AnalizadorRedUrbana	La lógica: leer y guardar archivos, y los algoritmos (Dijkstra, BFS y hubs). No tienen nada de interfaz.
`urbanroute.modelo`	GrafoTransporte, Estacion, Arista	Los datos de la red: el grafo, sus vértices (estaciones) y sus aristas (conexiones).
`urbanroute.estructuras`	TablaHashEstaciones, ListaEnlazada, Cola, Nodo	Estructuras de datos propias (no se usa Java Collections).

La caja gris **GraphStream** no es una clase del proyecto: es la librería externa que se usa **solo para dibujar** el grafo. Por eso aparece aparte, con el estereotipo «librería externa», y solo la usa la interfaz.

3. Qué hace cada clase

Main

Solo tiene el método `main`, que crea la ventana `InterfazUsuarioUI` («crea»).

InterfazUsuarioUI

Es la ventana del programa. Tiene tres atributos importantes: `grafo`, `gestorArchivo` y `analizador` (las tres asociaciones que salen de ella hacia abajo). Cada botón llama a uno de sus métodos privados: `cargarArchivo`, `guardarArchivos`, `agregarEstacion`, `mostrarRutaMasCorta`, etc. También tiene `dibujo` y `visor`, que son de GraphStream y sirven para mostrar el grafo en el panel central.

GestorArchivoCSV

Lee y guarda los archivos CSV. Al cargar, cada línea del archivo de estaciones se convierte en un vértice (`lineaAVertice`) y cada línea del archivo de conexiones en una arista (`lineaAArista`). Al guardar hace el camino inverso. No guarda el grafo como atributo: lo recibe como parámetro en cada método, por eso su relación con `GrafoTransporte` es una dependencia «carga y guarda». También crea objetos `Estacion` al leer el archivo («crea»).

AnalizadorRedUrbana

Tiene los tres algoritmos del proyecto: `rutaMasCorta` (Dijkstra), `buscarSubredes` (BFS) y `buscarHubs` (estaciones con mayor grado). Guarda una referencia al grafo, pero **solo lo lee**, nunca lo modifica. Usa `Estacion` para recorrer los vecinos y la `Cola` propia para el BFS.

GrafoTransporte

Es el grafo no dirigido con peso. Sus estaciones se guardan en una `TablaHashEstaciones` (composición: el grafo la crea y es el único que la usa). Ofrece los métodos para agregar y eliminar estaciones y conexiones, buscar una estación por ID y consultar vecinos y tiempos. Cuando se agrega una conexión, crea una `Arista` en cada extremo («crea»), porque el grafo es no dirigido.

Estacion

Es un vértice del grafo. Tiene sus datos (ID, nombre, línea, zona y capacidad) y su **lista de adyacencia**: `conexiones`, una `ListaEnlazada<Arista>`. Por eso hay dos composiciones: una hacia `ListaEnlazada` (la lista) y otra hacia `Arista` (0..* aristas guardadas en esa lista). `getGrado()` devuelve cuántas conexiones tiene.

Arista

Es una conexión hacia otra estación: guarda el ID del destino y el tiempo en minutos (el peso). No usa a ninguna otra clase.

TablaHashEstaciones

Tabla hash propia con encadenamiento separado. Su atributo `tabla` es un arreglo de listas enlazadas (composición hacia `ListaEnlazada`, empieza con 16 posiciones y por eso la multiplicidad es `16..*`). Guarda las estaciones (agregación «guarda», 0..*). Tiene la función hash (`funcionHash`) y el `rehash` que duplica la tabla cuando el factor de carga pasa de 0.75.

ListaEnlazada<T>, Cola<T> y Nodo<T>

Son las estructuras genéricas propias. `ListaEnlazada` (lista simple) y `Cola` están hechas de `Nodo` (composición). `Nodo` tiene una flecha hacia sí mismo llamada `siguiente` (0..1): cada nodo apunta al siguiente, o a nada si es el último. La `<T>` significa que sirven para cualquier tipo de dato: la misma lista guarda `Arista` en una estación y `Estacion` en la tabla hash.

4. Ejemplos: cómo se recorre el diagrama

Cargar un archivo. El usuario pulsa Cargar → `InterfazUsuarioUI.cargarArchivo()` → `GestorArchivoCSV.cargarArchivo(archivo, grafo)` → por cada línea, `lineaAVertice` crea una `Estacion` y la agrega con `GrafoTransporte.agregarEstacion` → el grafo la inserta en la `TablaHashEstaciones`, que la guarda en una de sus `ListaEnlazada` (formada por `Nodo`).

Agregar la conexión PRO-PER de 2 minutos. `InterfazUsuarioUI.agregarConexion()` → `GrafoTransporte.agregarConexion("PRO", "PER", 2)` → busca las dos estaciones en la tabla hash → crea `Arista(PER, 2)` en PRO y `Arista(PRO, 2)` en PER, y cada una se inserta en la lista `conexiones` de su estación.

Ruta más corta. `InterfazUsuarioUI.mostrarRutaMasCorta()` → `AnalizadorRedUrbana.rutaMasCorta(origen, destino)` → pide las estaciones al grafo con `obtenerEstaciones()` y recorre los vecinos de cada `Estacion` (`getIdVecino`, `getTiempoConexion`) → devuelve la ruta → la interfaz la muestra y la resalta en el dibujo con GraphStream.

Subredes. `AnalizadorRedUrbana.buscarSubredes()` → lanza un BFS desde cada estación no visitada usando la `Cola` propia → cada BFS es una subred.

5. Relación con las tarjetas CRC

Las relaciones del diagrama coinciden con las colaboraciones de las tarjetas CRC (y con las líneas "Colabora con / Es usada por" del Javadoc de cada clase):

Clase	Colabora con
InterfazUsuarioUI	GestorArchivoCSV, AnalizadorRedUrbana, GrafoTransporte (no usa Estacion, Arista ni la tabla hash)
GestorArchivoCSV	GrafoTransporte, Estacion
AnalizadorRedUrbana	GrafoTransporte, Estacion (y la Cola para el BFS)
GrafoTransporte	TablaHashEstaciones, Estacion, Arista
Estacion	Arista (en su lista de conexiones)
TablaHashEstaciones	Estacion; solo la usa GrafoTransporte
Arista	Ninguna

Un detalle importante del diseño: **la tabla hash solo la usa `GrafoTransporte`**. Las demás clases buscan estaciones a través del grafo (`buscarEstacion`), así el resto del programa no depende de

cómo se guardan.

6. Posibles preguntas y respuestas cortas

- **¿Por qué la interfaz no se conecta con Estacion ni con la tabla hash?** Porque todo pasa por el grafo, el gestor y el analizador; así la vista no depende de los detalles de los datos.
- **¿Por qué el gestor tiene dependencia y no asociación con el grafo?** Porque no guarda el grafo como atributo: lo recibe como parámetro en `cargarArchivo` y `guardar`.
- **¿Por qué el analizador sí tiene asociación?** Porque guarda el grafo en su atributo `grafo`, aunque solo lo lee.
- **¿Dónde está la lista de adyacencia?** En `Estacion.conexiones`: cada estación tiene su `ListaEnlazada<Arista>`.
- **¿Por qué Nodo tiene una flecha hacia sí mismo?** Porque el atributo `siguiente` es otro `Nodo`; así se encadenan las listas y la cola.
- **¿Dónde se usa GraphStream?** Solo en `InterfazUsuarioUI`, para dibujar. Los algoritmos y las estructuras son propios.